Les transformations aléatoires peuvent-elles neutraliser une perturbation universelle ?

Ce que dit la littérature, et ce qu'on peut en attendre pour notre projet

En bref

• **L'idée a été essayée** dès 2017-2018 : redimensionner, décaler, compresser ou bruiter l'image au hasard avant de la donner au réseau (Xie et al. ; Guo et al.). Contre des attaques qui ignorent la défense, les résultats sont spectaculaires (de 0 % à 97-99 % de précision).

• **Ces défenses ont toutes été cassées** par des attaques qui connaissent la transformation (Athalye, Carlini et Wagner, 2018). L'outil clé est l'**EOT** : optimiser la perturbation en moyenne sur toutes les transformations aléatoires. Résultat : 0 % de précision.

• Une **perturbation universelle** se prête particulièrement bien à l'EOT : l'Adversarial Patch (Brown et al., 2017) est universel **et** résiste aux rotations, changements d'échelle et positions aléatoires.

• La seule défense aléatoire **prouvée**, le randomized smoothing (Cohen et al., 2019), ne garantit la robustesse que pour des perturbations ℓ_2 de norme ≤ 0,5 à 3, alors que notre perturbation resnet50 a $\|v\|_2 = 10{,}46$.

• **Pour notre projet** : l'aléatoire affaiblira probablement notre perturbation actuelle (calculée sans en tenir compte), mais une perturbation recalculée avec EOT devrait retrouver son efficacité. C'est une expérience instructive, pas une défense.

1. Vocabulaire indispensable

Perturbation adversariale. Un petit vecteur r ajouté à une image x tel que le classifieur change d'avis : la classe prédite pour x + r diffère de celle prédite pour x. Elle est en général calculée pour une image. Une **perturbation universelle** v est un seul vecteur qui marche pour la plupart des images (notre sujet).

Défense par prétraitement aléatoire. On ne touche pas au réseau f ; on transforme l'entrée par une transformation t_θ tirée au hasard (θ = taille, position, bruit…) à chaque prédiction : le classifieur défendu est $f(t_\theta(x))$. L'espoir : la transformation « casse » la structure fine de la perturbation, et l'attaquant ne sait pas quelle transformation sera tirée.

Modèles de menace — ce que l'attaquant connaît. C'est le point qui explique tout le reste :

Modèle	L'attaquant connaît…	Commentaire
Boîte noire	rien du modèle ; il attaque un autre modèle et espère que ça se transfère	le plus faible
Boîte grise	le réseau et ses poids, **mais pas la défense**	cadre de Xie et Guo
Boîte blanche	tout, y compris la défense et la loi de l'aléatoire (mais pas le tirage)	
Attaque adaptative	boîte blanche + attaque **conçue spécialement contre cette défense**	standard exigé depuis 2018

Le principe de Kerckhoffs, emprunté à la cryptographie, s'applique : une défense doit résister à un attaquant qui la connaît. Une défense qui ne tient que parce que l'attaquant l'ignore est de la « sécurité par l'obscurité ».

2. Xie et al. (ICLR 2018) : redimensionnement et padding aléatoires

Article : Mitigating Adversarial Effects Through Randomization. C'est exactement l'idée « mettre un peu d'aléatoire dans chaque image ».

Méthode

- **Redimensionnement aléatoire** : l'image 299×299 est redimensionnée en r×r, avec r entier tiré dans [299, 331).
- **Padding aléatoire** : l'image redimensionnée est complétée par des zéros jusqu'à 331×331, à une position aléatoire.
- Cela fait **12 528** transformations possibles. Les deux couches sont ajoutées **au moment de la prédiction**, à des réseaux déjà entraînés, sans réentraînement.
- Testé sur Inception-v3, ResNet-v2-101, Inception-ResNet-v2 et une version entraînée de façon adversariale, contre FGSM, **DeepFool** et C&W.

Trois scénarios d'attaque, du plus faible au plus fort

- **Vanilla** : l'attaquant attaque le réseau seul, sans connaître la défense.
- **Single-pattern** : il attaque le réseau + une transformation fixe (image centrée, padding de 16 pixels, sans redimensionnement).
- **Ensemble-pattern** : il attaque un ensemble de 21 transformations prédéfinies.

Résultats (précision top-1 sur les images attaquées, sans défense → avec défense)

Modèle / attaque	Vanilla	Ensemble-pattern
Inception-v3, DeepFool	0 % → 98,3 %	0,6 % → 81,3 %
Inception-v3, C&W	0 % → 96,9 %	0,6 % → 62,9 %
ResNet-v2-101, C&W	0 % → 97,1 %	1,0 % → 74,3 %
Inception-v3, FGSM ε = 10	33,0 % → 52,4 %	—

La précision sur les images **propres** ne baisse que de 0,7 à 2,7 points. Au concours NIPS 2017, la défense combinée à un modèle entraîné adversarialement obtient un score normalisé de 0,924, contre 0,773 pour l'entraînement adversarial seul.

Explication et limites (selon les auteurs)

- **Pourquoi ça marche** : les attaques itératives (DeepFool, C&W) « collent » au réseau exact qu'elles attaquent ; leurs perturbations sont très fines et un décalage d'un seul pixel suffit à les casser. Dans leur diagnostic, **un padding d'un pixel** fait remonter la précision sous C&W de 0,8 % à 70,2 %, et **un redimensionnement d'un pixel** de 2,6 % à 97,8 %.
- **Limites reconnues** : moins efficace contre FGSM avec un grand ε (perturbation plus grossière) ; dès que l'attaquant moyenne sur plusieurs transformations (ensemble-pattern), la précision chute nettement (62,9 % sous C&W) ; impossible d'attaquer les 12 528 transformations à la fois, donc l'attaque la plus forte n'a pas été testée.

À retenir

La dernière limite est la plus importante : la défense a été évaluée contre des attaquants qui ne moyennaient que sur 21 transformations. La section 4 montre ce qui se passe quand l'attaquant moyenne correctement.

3. Guo et al. (ICLR 2018) : transformations de l'image

Article : Countering Adversarial Images using Input Transformations. Même idée, avec d'autres transformations sur un ResNet-50 :

Transformation	Aléatoire ?	Principe
Recadrage + redimensionnement	oui	moyenne des prédictions sur 30 recadrages aléatoires 90×90, remis en 224×224
Réduction de profondeur	non	chaque pixel codé sur 3 bits au lieu de 8
Compression JPEG	non	qualité 75
Variation totale (TVM)	oui	on garde chaque pixel avec probabilité 0,5, puis on reconstruit une image « lisse »
Image quilting	oui	chaque morceau 5×5 remplacé par un morceau d'image propre tiré parmi ses plus proches voisins

Résultats (boîte grise, distance ℓ_2 normalisée 0,06) : sous DeepFool, 44,9 % de précision avec le recadrage, 44,7 % avec TVM, 34,5 % avec le quilting, contre 1,8 % pour un modèle entraîné adversarialement. Combiner recadrage + TVM + quilting atteint 51,5 %. Le résumé annonce que la meilleure défense élimine 60 % des attaques fortes en boîte grise et 90 % en boîte noire.

Pourquoi ils pensaient que ça tiendrait : (1) TVM et quilting ne sont **pas dérivables**, on ne peut donc pas calculer de gradient à travers ; (2) l'**aléatoire** oblige l'attaquant à trouver une perturbation qui marche « pour toute la distribution d'images possibles ». Le point (2) est exactement ce que fait une perturbation universelle... ce qui n'est pas bon signe pour la défense.

4. Athalye, Carlini et Wagner (ICML 2018) : tout est cassé

Article : Obfuscated Gradients Give a False Sense of Security. L'article de référence pour comprendre pourquoi les défenses précédentes paraissaient solides.

Le diagnostic : des gradients « brouillés »

Les attaques (DeepFool, PGD, C&W) suivent le **gradient** du réseau. Beaucoup de défenses ne rendent pas le réseau plus robuste : elles rendent seulement le gradient **inutilisable**. Trois formes :

- **Gradients brisés** : une opération non dérivable (JPEG, quilting) ou numériquement instable ;
- **Gradients stochastiques** : le gradient dépend du tirage aléatoire, un seul tirage donne une mauvaise direction (**c'est le cas de ton idée**) ;
- **Gradients qui explosent ou s'annulent** : calculs très profonds, par exemple plusieurs passages dans un réseau.

Les deux armes

EOT (Expectation Over Transformation) — contre l'aléatoire. Au lieu d'attaquer $f(t_\theta(x))$ pour un tirage θ, on attaque la moyenne :

$$\text{maximiser } E_\theta \left[\text{perte}(f(t_\theta(x + r))) \right] \text{ avec } \nabla E_\theta[\dots] = E_\theta[\nabla \dots]$$

En pratique, à chaque pas, on tire plusieurs transformations, on calcule le gradient pour chacune et on **fait la moyenne**. La perturbation obtenue marche en moyenne sur toutes les transformations, donc quel que soit le tirage de la défense.

BPDA (Backward Pass Differentiable Approximation) — contre la non-dérivabilité. On garde la vraie transformation au passage avant, et on la remplace au passage arrière par une

approximation dérivable, souvent simplement l'identité (on fait « comme si » JPEG ne changeait rien).

Résultats

Défense	Attaque	Résultat
Xie et al. (aléatoire)	EOT sur la distribution des transformations, ε = 0,031 en ℓ_∞ (≈ 8/255)	**100 %** de réussite de l'attaque ciblée, précision **0 %**, même en exigeant que l'image soit trompée 10 fois sur 10
Guo et al. (transformations)	EOT pour le recadrage, BPDA pour JPEG et bits, EOT + BPDA pour TVM et quilting	**100 %** de réussite, précision **0 %** pour la meilleure défense

Au total, sur 9 défenses publiées à ICLR 2018, 7 reposaient sur des gradients brouillés ; 6 sont contournées complètement et 1 partiellement.

Leurs recommandations pour évaluer une défense

- Supposer que l'attaquant connaît l'architecture, l'entraînement et la défense ; seul le tirage aléatoire reste secret.
- Évaluer contre des **attaques adaptatives**, conçues après la défense et contre elle.
- Ne pas limiter artificiellement le calcul de l'attaquant.
- Signaux d'alerte : une attaque en un pas meilleure qu'une attaque itérative ; la boîte noire meilleure que la boîte blanche ; une attaque qui n'atteint pas 100 % même sans limite de taille.

5. Brown et al. (2017) : une attaque universelle qui résiste à l'aléatoire

Article : Adversarial Patch. Ce n'est pas une défense, mais c'est la preuve la plus directe pour notre question.

Un **autocollant** (« patch ») qu'on pose n'importe où dans une scène pour que le réseau prédise une classe choisie, par exemple « grille-pain ». Il est calculé par :

$$\hat{p} = \operatorname{argmax}_p \, E_{x,\, t,\, l} \left[\log P(\, \hat{y} \mid A(p, x, l, t)\,) \right]$$

où la moyenne porte sur des **images** x au hasard (universalité), des **positions** l au hasard et des **transformations** t au hasard (translation, mise à l'échelle, rotation) : c'est une variante de l'EOT. Imprimé sur une imprimante couleur et posé sur une table à côté d'une banane, il fait passer VGG16 de « banane » (97 %) à « grille-pain » (99 %).

Pourquoi c'est décisif pour ton idée

L'objectif du patch est exactement « universelle + robuste à des transformations aléatoires ». Si on peut l'atteindre pour un autocollant qui subit rotations, changements d'échelle et déplacements, on peut l'atteindre pour une perturbation universelle soumise à un redimensionnement ou un décalage aléatoires : il suffit d'ajouter la moyenne sur les transformations dans l'algorithme.

Les auteurs expliquent que le patch marche parce qu'il est plus « saillant » que les vrais objets : le réseau, entraîné à trouver l'objet principal, se focalise dessus. On retrouve l'idée de la **classe dominante** (notre « spider web »).

6. Gao et al. (NeurIPS 2022) : les limites de fond

Article : On the Limitations of Stochastic Pre-processing Defenses. Quatre ans plus tard, une analyse générale de ces défenses.

Constat 1 — la plupart n'ont pas assez d'aléatoire. Les auteurs refont les attaques **sans EOT**, avec une simple attaque PGD standard. Résultat : contre Xie et al., 200 itérations de PGD sans EOT suffisent pour **100 %** de réussite ; contre Guo et al., **99,0 %**. Autrement dit, l'aléatoire de ces défenses ne gênait presque pas l'attaquant. Même étude de cas avec une rotation aléatoire entre −90° et +90° : 100 % de réussite.

Constat 2 — un compromis inévitable entre robustesse et invariance. Pour mettre beaucoup d'aléatoire sans détruire la précision sur les images propres, il faut réentraîner le modèle pour qu'il devienne **insensible** à la transformation. Mais plus il y est insensible, plus la transformation laisse passer la perturbation, et moins elle protège. Les auteurs montrent ce compromis théoriquement et expérimentalement : ces défenses « protègent » en rendant le modèle plus variable, et doivent reprendre cette variabilité pour rester utiles.

Conclusion des auteurs : ces défenses n'apportent pas de robustesse intrinsèque ; il faut trouver d'autres façons d'utiliser l'aléatoire.

7. Cohen, Rosenfeld et Kolter (ICML 2019) : l'aléatoire avec garantie

Article : Certified Adversarial Robustness via Randomized Smoothing. La version rigoureuse de « ajouter du bruit ».

On remplace le classifieur f par un **classifieur lissé** g qui vote sur des copies bruitées de l'image :

$$g(x) = \operatorname{argmax}_c \, P(f(x + \varepsilon) = c), \; \varepsilon \sim N(0, \sigma^2 I)$$

Si la classe A gagne le vote avec probabilité au moins p_A et toute autre classe au plus p_B, alors **aucune** perturbation de norme ℓ_2 inférieure à

$$R = (\sigma / 2) \cdot (\Phi^{-1}(p_A) - \Phi^{-1}(p_B))$$

ne peut changer la prédiction (Φ^{-1} : inverse de la fonction de répartition gaussienne). C'est une **preuve**, valable contre tout attaquant, y compris adaptatif. Sur ImageNet :

Rayon ℓ_2 garanti	σ utilisé	Précision garantie	Précision sur images propres
0,5	0,25	49 %	67 %
1,0	0,50	37 %	57 %
2,0	0,50	19 %	57 %
3,0	1,00	12 %	44 %

Limites : le prix en précision est élevé (67 % au mieux, contre ~76 % pour un ResNet-50 standard) ; le modèle doit être entraîné avec du bruit gaussien ; la prédiction demande des milliers de tirages (100 000 pour certifier) ; la garantie ne porte que sur la norme ℓ_2.

Comparaison avec notre perturbation

Notre perturbation resnet50 a $\|v\|_\infty$ = 10/255 mais $\mathbf{\|v\|_2 = 10{,}46}$ (images dans [0, 1]). Les rayons garantis sont de 0,5 à 3 : **trois à vingt fois trop petits**. Le lissage aléatoire ne donne donc aucune garantie contre une perturbation universelle de cette taille. C'est le problème de fond : une perturbation imperceptible pixel par pixel (ℓ_∞) est grande en norme ℓ_2 en dimension 150 528.

8. Et pour les perturbations universelles en particulier ?

Dans l'article d'origine (Moosavi-Dezfooli et al., 2017), les auteurs testent une défense par **réentraînement** : VGG-F est réentraîné 5 époques sur des images auxquelles on ajoute, une fois sur deux, une perturbation universelle tirée parmi 10 précalculées. Le taux de tromperie d'une perturbation recalculée sur le modèle réentraîné passe de 93,7 % à **76,2 %**, et répéter la procédure n'améliore pas les choses. Conclusion des auteurs : une amélioration modeste, pas une protection.

Défenses dédiées. Borkar et al. (CVPR 2020) ne transforment pas l'image : ils **régénèrent certaines caractéristiques à l'intérieur du réseau**, celles qui sont les plus sensibles aux perturbations universelles, et rapportent qu'une défense entraînée contre une attaque universelle protège aussi contre d'autres. Je n'ai pu lire que le résumé de cet article : à vérifier avant de s'appuyer sur ses chiffres.

Je n'ai pas trouvé d'étude dédiée spécifiquement aux **transformations aléatoires contre des perturbations universelles** (au-delà d'une thèse dont je n'ai pu consulter que l'extrait). C'est une petite lacune que notre projet peut combler à son échelle.

9. Synthèse

Approche	Ce qui était annoncé	Face à un attaquant qui connaît la défense
Redimensionnement + padding aléatoires (Xie)	DeepFool, C&W : 0 % → 97-98 %	0 % (EOT) ; même PGD sans EOT réussit à 100 %
Transformations (Guo)	jusqu'à 51,5 % sous DeepFool	0 % (EOT + BPDA) ; PGD sans EOT : 99 % de réussite
Transformations très aléatoires	robustesse croissante avec l'aléatoire	compromis robustesse / invariance (Gao)
Lissage aléatoire (Cohen)	garantie prouvée	tient, mais seulement pour $\|r\|_2 \leq 0{,}5\text{-}3$, et coûte ~10-30 pts de précision
Réentraînement sur perturbations universelles	—	93,7 % → 76,2 % de tromperie : modeste

10. Ce qui marcherait, et ce qui ne marcherait pas, dans notre projet

Ce qui marcherait probablement

- **Affaiblir nos perturbations actuelles.** Elles ont été calculées sans aucune transformation, et DeepFool ne franchit les frontières que de 2 %. Nous avons déjà observé leur fragilité : sur Imagenette, le simple écrêtage dans [0, 1] faisait passer le taux sur X de 85,6 % à 51,2 %. Xie et al. montrent qu'un décalage d'un pixel suffit souvent à casser une perturbation DeepFool. Un redimensionnement ou un décalage aléatoire devrait déplacer la « toile d'araignée » par rapport à la grille de pixels et faire nettement baisser le taux de tromperie.
- **Avec une réserve** : la perturbation resnet50 sur ImageNet est plus « grossière » (texture répétée sur toute l'image, $\|v\|_2 = 10{,}46$), elle pourrait mieux résister qu'une perturbation DeepFool d'une seule image. C'est à mesurer.

Ce qui ne marcherait pas

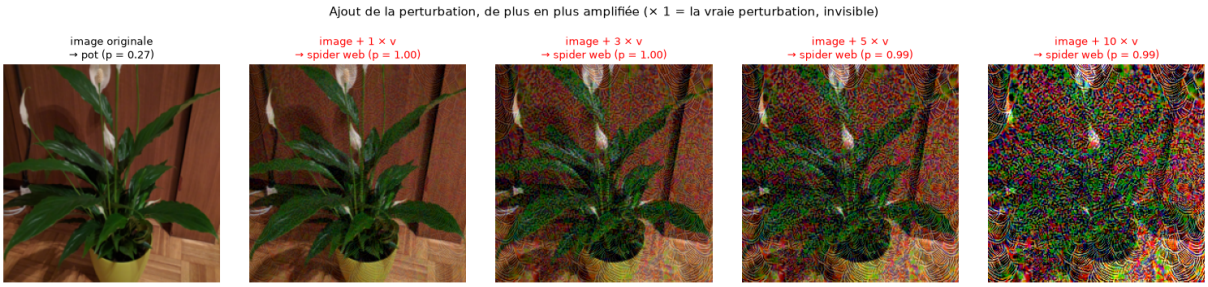
- **Comme défense réelle.** Il suffit de recalculer la perturbation universelle en **moyennant sur les transformations aléatoires** (EOT). C'est particulièrement facile ici : l'algorithme universel moyenne déjà sur des milliers d'images ; ajouter un tirage de transformation à chaque image ne change presque rien. L'Adversarial Patch montre que le résultat reste efficace.
- **Avec beaucoup plus d'aléatoire** : la précision sur les images propres chute, et réentraîner le modèle pour la récupérer réduit la protection (Gao).
- **Avec du bruit gaussien « garanti »** : les rayons certifiés sont bien trop petits face à $\|v\|_2 = 10{,}46$.

L'expérience que nous pourrions faire

Elle ne donnera pas une nouvelle défense, mais elle reproduira, sur nos propres perturbations, la leçon de la littérature :

Étape	Ce qu'on fait	Ce qu'on mesure	Attendu
1. Défense naïve	évaluer nos perturbations après redimensionnement + padding aléatoires, bruit gaussien, flou, JPEG	fooling rate ; précision sur images propres	fooling rate en nette baisse, précision presque intacte
2. Riposte EOT	recalculer la perturbation universelle en appliquant une transformation aléatoire à chaque image de l'algorithme	fooling rate **avec** la défense	retour à un fooling rate élevé
3. Prix de la défense	augmenter l'intensité de l'aléatoire	précision propre vs fooling rate EOT	il faut sacrifier beaucoup de précision

Ce qu'il faut écrire dans le code : une option de transformation aléatoire dans `evaluate.py` (étape 1) et une option EOT dans `universal_pert.py` (étape 2). L'étape 1 peut tourner sur un PC avec Imagenette ; l'étape 2 demande un GPU.



Notre perturbation universelle resnet50, ajoutée à une image et amplifiée (× 1 = la vraie perturbation, invisible). C'est cette structure fine (la « toile d'araignée ») qu'une transformation aléatoire pourrait déplacer ou brouiller.

Références

[1] C. Xie, J. Wang, Z. Zhang, Z. Ren, A. Yuille. Mitigating Adversarial Effects Through Randomization. ICLR 2018. https://arxiv.org/abs/1711.01991

[2] C. Guo, M. Rana, M. Cisse, L. van der Maaten. Countering Adversarial Images using Input Transformations. ICLR 2018. https://arxiv.org/abs/1711.00117

[3] A. Athalye, N. Carlini, D. Wagner. Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples. ICML 2018. https://arxiv.org/abs/1802.00420 — code : https://github.com/anishathalye/obfuscated-gradients

[4] T. B. Brown, D. Mané, A. Roy, M. Abadi, J. Gilmer. Adversarial Patch. 2017. https://arxiv.org/abs/1712.09665

[5] Y. Gao, I. Shumailov, K. Fawaz, N. Papernot. On the Limitations of Stochastic Pre-processing Defenses. NeurIPS 2022. https://papers.neurips.cc/paper_files/paper/2022/hash/997089469acbeb410405e43f0011be1f-Abstract-Conference.html

[6] J. Cohen, E. Rosenfeld, J. Z. Kolter. Certified Adversarial Robustness via Randomized Smoothing. ICML 2019. https://arxiv.org/abs/1902.02918

[7] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, P. Frossard. Universal Adversarial Perturbations. CVPR 2017. https://arxiv.org/abs/1610.08401

[8] T. Borkar, F. Heide, L. Karam. Defending Against Universal Attacks Through Selective Feature Regeneration. CVPR 2020. https://openaccess.thecvf.com/content_CVPR_2020/papers/Borkar_Defending_Against_Universal_Attacks_Through_Selective_Feature_Regeneration_CVPR_2020_paper.pdf